



Application Development DICT312

Week 1 - Introduction to Jakarta EE and Cloud Computing

Dr. O.S Ogunleye
Email: olalekan.ogunleye@ump.ac.za
Telephone: 013 002 0227
Office: Building 5 (Room 212)
Consultation(By Appointment Only)



Reading Materials

The following are the prescribed textbooks:

- Title: Jakarta EE for Java Developers: Build Cloud-Native and Enterprise Applications Using A High-Performance Enterprise Java Platform, 1st Edition. Author(s): Rhuan Rocha, Year of Publication:2022, Publisher: BPB Publications ISBN: 978-93-55510-082

The following are the recommended textbooks:

- Title: Design Patterns and Best Practices in Java, Edition: 1st. Author(s): Torje L, Ianculescu A and Singh K, Year of Publication: 2018, Publisher: Packt Publishing, ISBN: 978-1-78646-359-3



Introduction

The Java ecosystem has been used in for many years and has provided several kinds of solutions to any environment.

One of these environments is the enterprise environment, which has its necessities and challenges to be overcome.

Therefore, the Java ecosystem, promoted a project called J2EE, which is a project to provide well-known solutions to the common enterprise problems.

Over time, its name changed to Java EE, and now it is Jakarta EE, which is maintained by the Eclipse Foundation now.

In this session you will be shown what the Jakarta EE is and what are its goals.

Throughout this session, I will walk you through each main point of this project, analyzing the why for some changes and what can we expect from Jakarta EE. Besides that, we'll show you what a server application is, what its importance to the Jakarta EE ecosystem is, and what we need to work with Jakarta EE.



What is Jakarta EE?

Jakarta EE is a set of software components, APIs, for developing specifically enterprise Java applications.

These components are often referred to as specifications.

Jakarta EE specifications extend Java SE - the standard edition Java programming language - with ways to perform the functions particularly useful for an enterprise application.



Components of Jakarta EE Specification

Each Jakarta EE specification consists of a document defining the purpose of the API and how it's used; the API JAR, the actual interfaces to be coded against as a developer; and Technology Compatibility Kit (TCK), the test suite that is used to determine whether independent implementations of the API meet its requirements.



The Jakarta EE Runtime

The Jakarta EE specifications are designed to work with a Jakarta EE compatible runtime.

A runtime is a program which runs your application and handles the HTTP requests that connect it to its users on the internet.



Have you heard about Java EE?

Java EE is what Jakarta EE was formerly known as, when it was created by Sun Microsystems and then developed further by Oracle.

Development and progression of the specifications have now been taken on by open-source software organisation the Eclipse Foundation, but they did not have the legal rights to the javax namespace. Therefore, the transition has now been made to Jakarta EE.





Jakarta EE Makes Life Easier

Jakarta EE specifications avoid you having to write lots of extra code yourself.

As a result, it allow you as a developer to focus on the business logic of your application, rather than infrastructural tasks.

This doesn't just save time: your application is now actually separated from those infrastructural tasks in the runtime.

Complexity is hidden, and your experience as a developer will be better.

If you choose a managed runtime, you can also access benefits like high availability, clustering capabilities and the ability to store data from one node and retrieve it from another node.

All these qualities will make a more efficient system for your users.



Jakarta EE Specifications

Jakarta RESTful Web Services - support in web transfer over HTTP, according to the Representational State Transfer (REST) architectural pattern. It is often used to connect the front-end (user facing) with the back-end (connection to databases, data processing).

Jakarta Context and Dependency Injection (CDI) - provides features that allow for dependency injection - an object in your code receiving other objects it depends on. With it, you can define beans (objects that can be used with others) that provide services or keep user data, and define the way they will work, their lifecycle etc.

Jakarta Persistence - specifications about the mapping between your database tables and Java classes.

Jakarta Concurrency- allows you to retain context - for example security context - between threads.



Cloud Computing

Cloud Computing is a set of techniques and approaches to serving a computational resource as a service with a better and easier way for the end-user.

With cloud computing, we can have only a computational resource — such as memory, disk, CPU, node amount — that we really need and can scale up and down when needed.

With this, the business can save money and generate a faster and more effective response to the increasing demand.

Using cloud computing, we can have many types of services like Platform as a Service (PaaS), Infrastructure as a Service (IaaS), Software as a Service (SaaS), Function as a Service (FaaS), and others.



Benefits of Cloud Computing

Cloud computing provides many benefits to the business, which are as follows:

Pay per use: The end-user pays only for the resources used. It can save a lot of money for the business.

Resilience: The environment can recover after a failure.

Self-service provisioning: The computational resource can be scaled up according to the demands in an easier and faster way, using an interface, eliminating the traditional need for the IT administrators to provision and manage to compute resources.

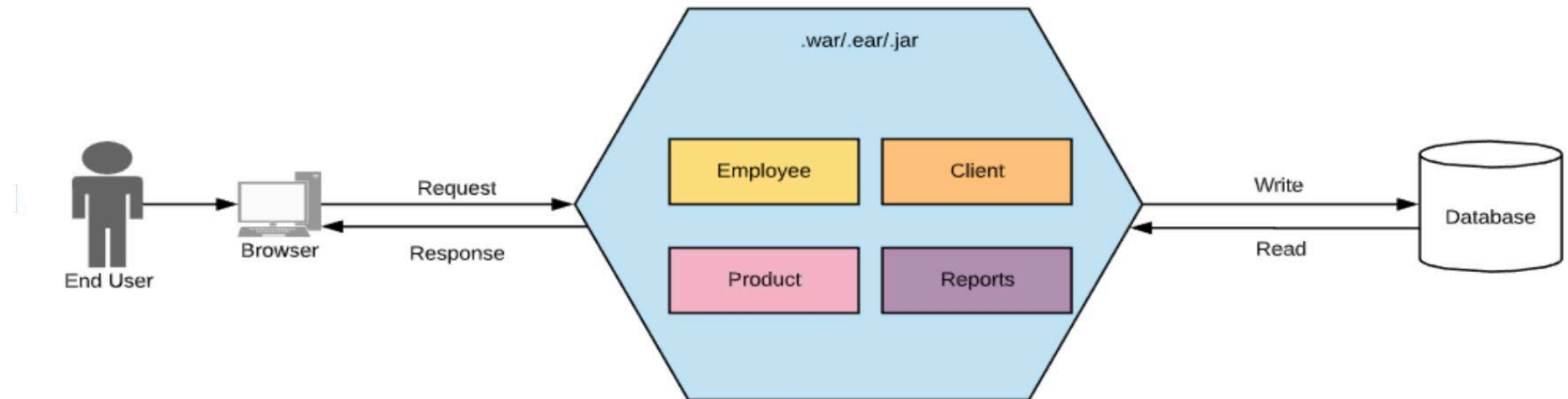
Elasticity: The computational resource can be scaled up or down at any time.

Fault Tolerance: As the environment can be run in many local areas, across many global regions, this environment has more fault tolerance. Clearly, cloud computing comes with many benefits to the business world, but to take advantage of this, the application needs to be prepared to work in the cloud computing approach. But what does it mean?



Monolith Architecture Vs Cloud Computing

The monolith architecture has worked very well and works very well today, but outside a cloud computing environment. The monolith is not adequate to take advantage of cloud computing, because if you need to scale up this application, you'll scale all monolith, that is, you will scale all business domains inside monolith, and the benefit of pay only for the resources used will be lost. The resolution of this problem was rethinking how to architect the applications.

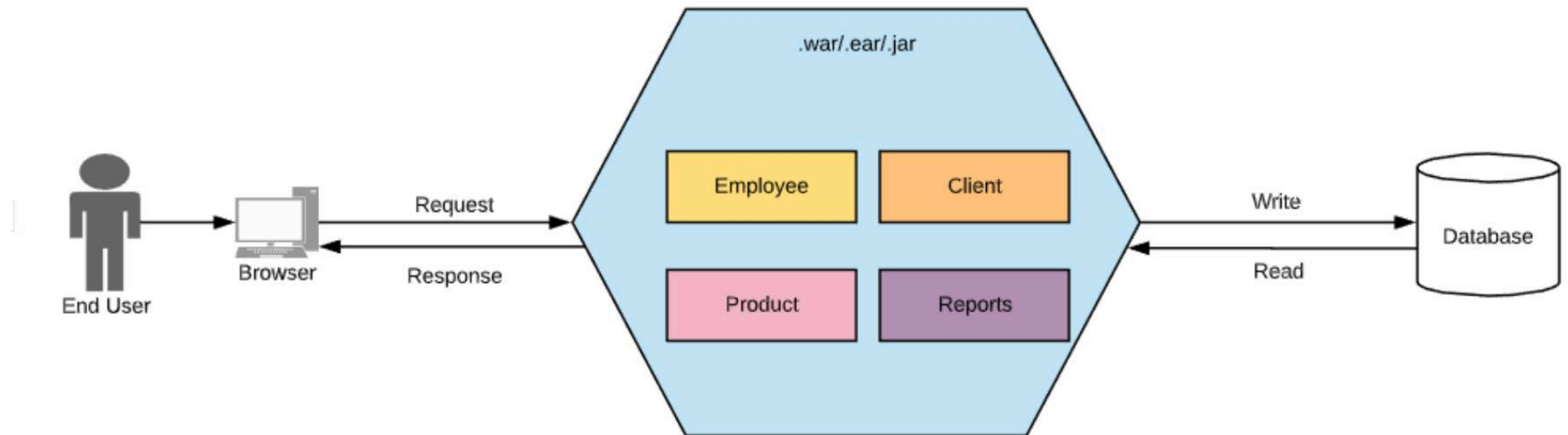




Monolith Architecture Vs Cloud Computing

The old approach used for a long time in the technological world has used a monolith architecture — all business domains or many business domains inside the same application — to develop the application.

The result is a big application that has many responsibilities. The Figure below illustrates this as follows:





Microservice Architecture

Rethinking about the architecture, the software engineers and architects noticed that to take advantage of cloud computing, they will have to break the monolith into the smallest parts with fine granularity to permit scaling only the part that really needs to be scaled.

But how can it be broken? What are the metrics to define the boundaries of each part?

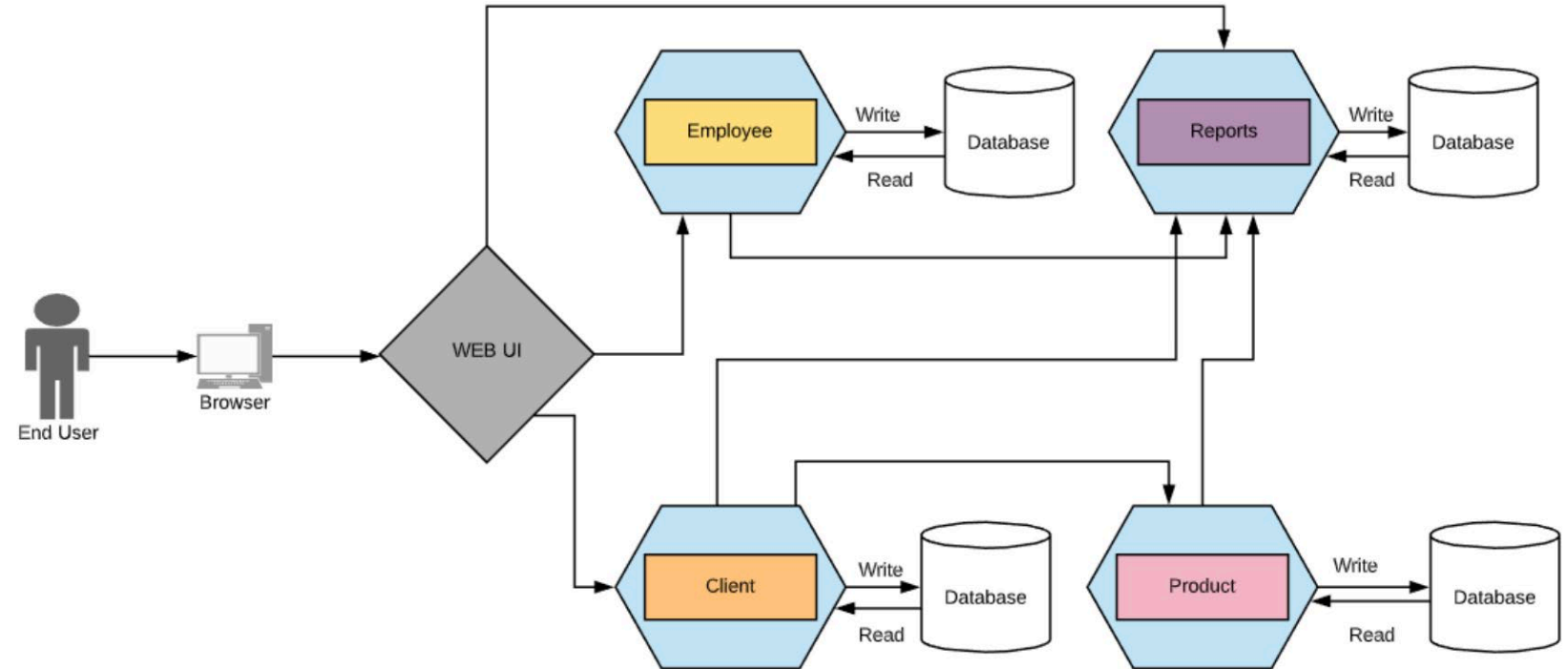
The microservice architecture breaks the monoliths into many small parts called microservice, which generally are broken according to the business domain, that is, each microservice has the responsibility to care about one business domain.

Therefore, the business domain is used to define the boundaries of the microservice.



Microservice Architecture

The use of the business domain as a base to define the microservices boundary is the approach widely used, but you can use another according to your scenario. But keep in mind that you should decompose to improve the ability to scale. Figure 1.2 illustrates this architecture as follows:



Microservice Architecture



In a microservice architecture, each microservice has low-coupling with each other, and they are independent and deployable.

This means that microservices are a small application that runs without the need for another microservice to do that.

Those microservices integrate with each other when needed, to provide a response to the end-user.

It is very good, but now the software engineers and architects have other problems:



Microservice Architecture

How will those microservices be managed and how will those applications be integrated with each other without becoming chaos?

How to discover a new microservice when a new microservice service is born? How to load-balance those microservices and how will those microservices be scaled up and down without a side effect?



Microservice Architecture

How will those microservices be managed and how will those applications be integrated with each other without becoming chaos?

How to discover a new microservice when a new microservice service is born? How to load-balance those microservices and how will those microservices be scaled up and down without a side effect?

This will lead us to something else – The Twelve Factors

THANK YOU



**UNIVERSITY OF
MPUMALANGA**